BSc Honours Viva Defense — Simplified Speaker Script (22 Slides)

Candidate: Thabang Nhlokoma Buthelezi (230011908)
Supervisors: Prof. Pragasen Mudali (UNIZULU) & Mr. Skhumbuzo Zwane (CSIR)
Dissertation: *Decision-Grade Multi-Objective Benchmarking of AI and Heuristic Controller Placement in Multi-Site Software Defined Networks* (University of Zululand, 2026)

Slide 1: Title Slide & Acknowledgements

Good morning Prof. Mudali, Mr. Zwane, and members of the examination panel. [Pause] My name is Thabang Nhlokoma Buthelezi, and today I am presenting my Honours mini-dissertation in Computer Science at the University of Zululand. Before I begin, I want to acknowledge my supervisors, Prof. Mudali and Mr. Zwane, for their guidance, as well as my family and the Department of Computer Science for their support. In simple terms, my research asks one practical question: when a large network uses Software Defined Networking, we have to choose where to place the smart 'brains'—called controllers—across the network. Today, many researchers say Artificial Intelligence and Deep Reinforcement Learning should make that decision. My study tested AI head-to-head against classic, simple math-based rules across 300 experiments, real Linux network emulation, and 1,000 link failures—to find out when AI is worth the effort, and when simple rules actually work better.

Slide 2: Presentation Roadmap & Research in 30 Seconds

Let me give you a quick map of today's talk—and the whole story in 30 seconds. My presentation follows all five chapters of my signed dissertation plus the appendices. First, in Chapters 1 and 2, I will explain what Software Defined Networking is, why choosing controller locations is tricky, and the gap we spotted in recent 2021-to-2026 papers. Second, in Chapter 3, I will show how we built a fair testbed comparing five algorithms across six networks. Third, in Chapter 4, I will share what our numbers actually proved—both on static networks, inside real Mininet Linux emulation, and when links keep failing. Finally, in Chapter 5, I will give engineers a clear, practical guide on which tool to pick. Here is the main headline before we even start: on normal, stable networks up to 50 switches, classic algorithms beat or match Deep Reinforcement Learning without wasting 11 to 21 seconds on neural training. AI only earns back its training time when the network changes repeatedly—between 59 and 208 times.

Slide 3: Chapter 1 — What is SDN & Why Placement Matters?

Chapter 1 sets the stage. In traditional networks, every router makes its own forwarding decisions, which makes wide-area management slow and messy. Software

Defined Networking—or SDN—fixes that by separating the 'brains', called controllers, from the 'roads', which are the switches. Whenever a switch sees unfamiliar traffic, it asks its controller what to do. Now, imagine you run a national network across 20 or 50 cities. You can think of placing controllers just like placing ambulance or fire stations across a country: if you only put one station in one city, everyone far away waits too long—and if a bridge washes out, whole towns get cut off. So we deploy k controllers across the network. That choice controls three things we measure in this study: first, Speed—how many milliseconds messages take to travel; second, Backup Safety—if one fiber cable breaks, what percentage of switches stay connected; and third, Compute Time—how long the algorithm takes to find the answer.

Slide 4: Chapter 1 — The Problem: 4 Blind Spots in Prior Research

In Section 1.3 of Chapter 1, we identified four major blind spots in existing research. First, many recent AI papers report that a trained Deep Q-Network picks controller locations in under a millisecond—which sounds amazing—but they completely hide the 11 to 21 seconds of training time required beforehand. Comparing an already-trained AI to a classic algorithm that solves the problem from scratch in 5 milliseconds isn't a fair comparison! Second, many studies run only one random seed on one map, instead of running balanced multi-seed tests. Third, optimizing only for speed can leave switches stranded behind a single fragile bridge cable. And fourth, because of these gaps, network engineers have had no clear decision rule for when to use simple heuristics and when to actually use AI.

Slide 5: Chapter 1 — Our Goal, 7 Objectives & Significance

Section 1.4 defines our main goal and seven specific objectives. To keep them easy to follow, our seven objectives fall into three clear stages. First, Stage A—Objectives 1 and 2: build a clean, automated Python pipeline and implement all five algorithms under the exact same rules. Second, Stage B—Objectives 3, 4, and 5: verify that our Deep Q-Network actually learns steadily across 10 independent runs, run a balanced 300-experiment comparison across different network shapes, and map the 3D trade-offs between speed, reliability, and compute time. Third, Stage C—Objectives 6 and 7: double-check our rankings inside a live Mininet OpenFlow emulator, measure dynamic link failures, and turn everything into a clear decision chart for network engineers up to 50 switches.

Slide 6: Chapter 2 — How SDN Works & Why Placement Is Hard

Turning to Chapter 2, Section 2.1 explains how SDN splits a network into three layers: the Data Plane where switches move packets, the Control Plane where controllers act as the central brains, and the Application Plane for network policies. Whenever a switch gets a new type of traffic, it sends a 'Packet-In' request to its controller and

waits for instructions. Now, Section 2.2 explains why placing those controllers is mathematically hard—in computer science, it is NP-hard. On a 20-node network with 3 controllers, there are 1,140 combinations. Jump to 50 nodes and 5 controllers, and there are over 2.1 million combinations! Because brute-force checking doesn't scale, engineers rely on smart algorithms to optimize either average delay or worst-case delay.

Slide 7: Chapter 2 — The 3 Algorithm Families & Link-Cut Safety

Sections 2.3 through 2.6 review the three families of placement algorithms. First are classic heuristics: Greedy K-Center spreads controllers far apart to protect edge switches, while Spatial K-Means clusters nearby cities together, just like choosing regional warehouse hubs. Second is evolutionary search, specifically a Genetic Algorithm that tests a population of solutions and combines the best ones over generations. Third is Deep Reinforcement Learning—like Deep Q-Networks—which tries placements, gets a reward score, and updates neural weights over hundreds of episodes. Section 2.6 highlights a critical real-world check: single-link survivability. If a graph has 'bridge links'—meaning only one cable connects a branch of the network—cutting that one cable isolates those switches from their controller.

Slide 8: Chapter 2 — What Was Missing (2021–2026 Literature Gap)

Section 2.7 summarizes 2021-to-2026 studies in Table 2.1. Look how easy this is to read: prior studies each looked at one or two pieces of the puzzle. Classical works like Mohanty (2021) checked speed, and Wang and Chen (2021) checked link failures. Newer AI papers like Benoudifa (2023), Kurra (2025), and Farahi (2026) focused on neural networks—but NOT A SINGLE ONE cleanly separated offline training time from online inference speed, while also testing single-link survivability and real 5-way Mininet emulation together. Our study fills all four columns at once.

Slide 9: Chapter 3 — Our Study Design & Two Clear Hypotheses

Moving to Chapter 3, Figure 3.1 shows our research workflow. In the top-right box, our testbed loops through real and synthetic networks for 2, 3, and 5 controllers—with built-in CPU thermal cooldown pauses so hardware heat never distorts timing measurements. Notice the badge at the top: 'No cross-seeding.' That means we never feed classic answers to the AI as a cheat sheet; the AI learns completely from scratch on the green track while classic heuristics run cleanly on the blue track. Both tracks feed into Phase 3 statistics and Phase 4 recommendations at the bottom. We tested two simple hypotheses: first, whether AI beats heuristics at all; and second, whether AI's training time makes it less practical on static networks.

Slide 10: Chapter 3 — The 6 Networks We Tested (Table 3.1)

Section 3.2 describes the six networks we tested in Table 3.1. First, we included two real-world wide-area backbones: Internet2, a sparser US research network, and ATT-MPLS, a tightly meshed commercial carrier network with plenty of alternate routes. Then, to test scale safely, we generated two synthetic network families at both 20 and 50 nodes. Barabási-Albert—or BA—creates 'hub-and-spoke' networks just like major airlines, where a few busy hubs connect everyone. Waxman creates flat geographic meshes like city roads, where switches connect mostly to nearby neighbors. Testing both shapes is crucial because what works on a hub-and-spoke network can struggle on a flat geographic map.

Slide 11: Chapter 3 — Meet the 5 Algorithms & System Design

Section 3.3 and Figure 3.2 show our system architecture and the five algorithms we built inside one unified Python codebase. First is Random, which simply guesses locations to give us a baseline floor. Second is Greedy K-Center, which spreads controllers as far apart as possible so no edge switch is left stranded. Third is Spatial K-Means, which splits the map into balanced regions and picks the center node of each zone. Fourth is our Genetic Algorithm, which evolves candidate placements over generations. Fifth is our Deep Q-Network—built in PyTorch using the exact 2026 AP-DQN neural architecture of Farahi et al., with two 64-neuron hidden layers and a composite reward scoring both speed and backup safety.

Slide 12: Chapter 3 — Fairness Controls, Caching & Ethics (Sec. 3.4–3.6)

Sections 3.4 through 3.6 explain how we kept every test fair, fast, and scientifically rigorous. We scored every placement on three axes: delay speed, single-link failure survivability, and compute time—always separating offline training from online execution. For our main factorial test in Table 3.3, we ran 5 algorithms across 2 network types, 2 sizes, 3 controller budgets, and 5 random seeds, giving 300 controlled runs. Because checking every single-link cut across thousands of AI training episodes is computationally heavy, we engineered a SHA-256 topology cache that made training 46.4% faster while matching exact math to 9 decimal places. Finally, 22 automated unit tests verify our code, and everything uses public and synthetic graphs with zero human-subject risks.

Slide 13: Chapter 4 — Did Our AI Actually Learn? (1,000-Episode Proof)

Now we move into Chapter 4—our actual results! A fair question any examiner might ask is: 'If classic algorithms beat AI, did your AI actually learn properly?' Figure 4.1 proves that it did. Over 1,000 episodes on the Internet2 backbone, our PyTorch Deep Q-Network learns rapidly, stabilizing in about 20 episodes and climbing to a final average score of 13.2 out of 15, with a peak score of 14.6—giving 1.42 milliseconds delay and 100% single-link reachability. So the AI definitely learned! But look at the

time cost: learning those 1,000 episodes took 21.26 seconds without caching—or 11.40 seconds with our SHA-256 cache—just so a single decision afterward could run in 0.029 milliseconds.

Slide 14: Chapter 4 — 10-Run AI Stability Check (No Cherry-Picking)

In AI research, people sometimes cherry-pick one lucky random seed that looked good. To avoid that completely, Figure 4.2 and Table 4.1 audit our Deep Q-Network across 5 different random seeds on both Internet2 and ATT-MPLS—a total of 10,000 training episodes. Every single run converged smoothly with zero crashes or divergent behavior, scoring consistently around 14.2 on Internet2 and 14.85 on ATT-MPLS. That proves our AI setup is completely reliable and fair before we pit it against the classic algorithms.

Slide 15: Chapter 4 — Static Results: Hub-and-Spoke Maps (BA)

Now look at what happened when we tested static 'hub-and-spoke' Barabási-Albert networks in Table 4.2 and Figure 4.3. Two clear winners stood out. First, our Genetic Algorithm achieved the lowest average delay in all 6 out of 6 test cells, cutting latency by up to 39.3% compared to Random. Second, Greedy K-Center proved to be the superstar for instant speed: because BA networks have a few major central hubs, Greedy naturally grabs those hubs in under 0.21 milliseconds with zero training time! Meanwhile, Spatial K-Means occasionally stumbled when k=2 because flat 2D clusters miss how central hubs bridge across the graph. Importantly, because BA networks have redundant paths, every algorithm kept 100% reachability under single-link failures.

Slide 16: Chapter 4 — Static Results: Flat Geographic Maps (Waxman)

On flat, geographic Waxman networks—shown in Table 4.3 and Figure 4.4—the story flips between our fast heuristics! Here, links connect nearby neighbors like streets on a map. Because distances match physical space, Spatial K-Means shines—coming within a hair of the Genetic Algorithm's delay in just 3 to 8 milliseconds! Once again, GA won lowest average latency across all 6 cells, cutting delay by up to 55.3% over Random. One huge practical finding on Waxman maps: unlike well-meshed backbones, Waxman graphs often have fragile 'bridge links' leading to outer nodes. When one of those bridge cables breaks, reachability drops from 100% down to between 94.5% and 98.8%—proving why single-link failure checks must never be ignored.

Slide 17: Chapter 4 — The Big Verdict: 300-Run Statistics & 3D Pareto

Slide 17 brings all 300 synthetic runs together using formal statistics in Table 4.4b and our 3D trade-off chart in Figure 4.5. The Multi-Factor ANOVA confirms with over

99.99% confidence that algorithm choice, network topology, network size, and controller budget all strongly affect latency. Look at the crucial statistical test between Deep Q-Networks and simple K-Means on static maps: p = 0.68. In plain English, that means there is NO statistically significant difference in latency between our trained AI and simple K-Means! When you plot Speed, Reliability, and Total Compute Time together in 3D in Figure 4.5, classic heuristics and GA sit right at the front of the optimal frontier with zero training tax, leaving AI dominated on static networks—confirming our Alternative Hypothesis H11.

Slide 18: Chapter 4 — Real Linux Emulation Test (Mininet & OVS)

Next, in Section 4.1.5, we tested whether our math rankings hold up in a real Linux network emulator using Mininet and Open vSwitch on Internet2. As Table 4.5 shows, the real round-trip ping times matched our Dijkstra rankings in exact order—a perfect +1.00 correlation! GA finished first at 2.72 milliseconds average RTT, Spatial K-Means came second at 2.85 milliseconds and achieved the lowest worst-case tail delay at 5.76 milliseconds, while both beat RL/DQN at 3.06 milliseconds. We also ran a '2-Second Gate Test' in Table 4.6 to explain why we trained the AI on graphs first instead of inside Mininet: tearing down Linux virtual switches after each episode took nearly 2 seconds—69% of the entire cycle! Training 1,000 episodes directly inside Mininet would spend 48 minutes per seed mostly restarting bridges instead of learning.

Slide 19: Chapter 4 — What If Links Keep Failing? (1,000-State Churn)

Section 4.1.6 asks the next natural question: what if the network isn't static—what if fiber links keep breaking and recovering over time? Table 4.6B reports our 1,000-state dynamic churn experiment across 100 sequential link cuts on Internet2 and ATT-MPLS. Here, we pre-trained our PyTorch DQN once and let it update placements via a single neural forward pass in just 0.021 to 0.026 milliseconds—without retraining—while GA and K-Means recomputed from scratch each time. Here is the exact break-even answer: AI finally pays back its training cost after 59 to 65 link changes compared to GA, and 146 to 208 changes compared to K-Means. However, notice two catches: Greedy K-Center is still fast at 0.05 milliseconds without any training, and because the basic DQN only looks at node degrees, GA still gives much lower latency after a failure.

Slide 20: Chapter 4 — Simple Decision Guide for Engineers (N ≤ 50)

Section 4.2 turns all our numbers into Figure 4.6 and Table 4.7: a simple four-rule decision guide for network engineers managing topologies up to 50 nodes. Rule 1: if your network is a stable flat geographic mesh, use Spatial K-Means for fast setup and the best worst-case tail delay. Rule 2: if your network is a stable hub-and-spoke core and you need an instant answer, use Greedy K-Center in under a quarter of a millisecond. Rule 3: if you are planning infrastructure offline and want the lowest

possible average delay, run our Genetic Algorithm. Rule 4: save Deep Reinforcement Learning for high-churn networks that change topology more than 60 to 208 times without retraining—and even then, check if simple Greedy K-Center already satisfies your delay budget!

Slide 21: Chapters 4 & 5 — Limitations, Contributions & Future Work

Wrapping up with Section 4.3 and Chapter 5: it is important to be transparent about our boundaries. All conclusions are strictly bounded to topologies up to 50 switches, single-link outages, and a fixed-size MLP neural architecture that cannot generalize zero-shot to a bigger graph without retraining. Even within those boundaries, our contributions are clear: we showed why separating training time from inference speed changes the verdict completely, proved how topology geometry picks the winning heuristic, and built a reproducible decision framework. For future Master's research in Section 5.4, the next logical step is replacing the fixed MLP with inductive Graph Neural Networks to scale to 100 or 500 switches with live traffic telemetry.

Slide 22: References, Appendices (A–C) & SATNAC 2026 Outputs

Finally, Slide 22 points to the supporting materials in the back of the signed dissertation: our fully updated 2021-to-2026 peer-reviewed reference list on pages 40 to 41, Appendix A with our exact software and seed manifest, Appendix B containing all 60 cells of raw benchmark numbers in Table B.1, and Appendix C with the Mininet emulation code. Beyond the dissertation itself, we also produced a 7-page IEEE conference paper targeting SATNAC 2026 and an official UNIZULU Research Poster. Thank you very much, Prof. Mudali, Mr. Zwane, and members of the examination panel—I am now ready for your questions.
